COMP1521
Wk2 Tutorial

Debugging MIPS

Getting Set Up

To get C-like syntax highlighting and debugging features, install the "Mipsy Editor Features" extension in VSCode!

Also, highlights syntax errors + hovering over most instructions will give a dropdown explaining what they do!

```
PRINT_STRING = 4


        .text
main:
        li      $v0, PRINT_STRING
        la      $a0, hello_str
        syscall                         # printf("Hello everyone! :)\n");

        li      $v0, 0
        jr      $ra                     # return 0;


        .data
hello_str:
        .asciiz "Hello everyone! :)\n"
```

addi: Adds an immediate value to the value in $Rs, stores result in $Rt

```
addi    $t1, $t0, 3
```

```
li      $t0, 1          # syntax is wrong: addi expects format
addi    $t1, $t0, $t0   # $destination register, $source register, constant!
```

Debugging Mode (1/3)

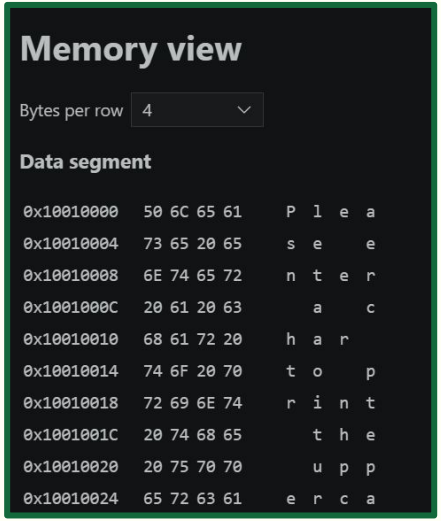
To start: click the icon in the top-right of the screen. (Note: not all features are fully functional, I'm just showing the ones that work!)

Run the code normally (all inputs/outputs in debug console launched at the bottom)

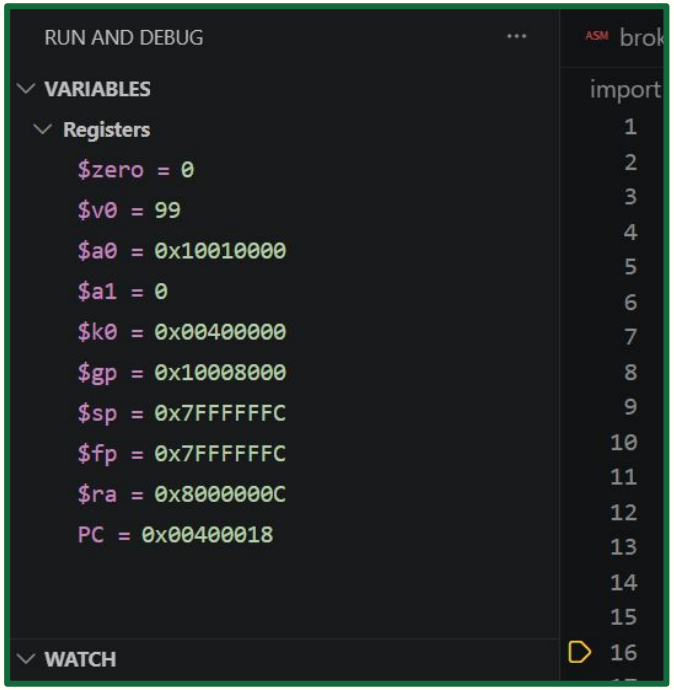
Step through the code line by line, forward and back

Stop the debugging mode

Opens view of the .data section (more relevant next week!)



Debugging Mode (2/3)



Variables Tab

- Lets you monitor the contents of registers during execution!
- Good for spotting if, e.g, you're incrementing weirdly in a loop or writing to the wrong register

Debugging Mode (3/3)

```
main:
        li      $v0, PRINT_STRING
        la      $a0, enter_char_str
        syscall

        li      $v0, READ_CHAR
        syscall
        move    $t0, $v0

        sub     $t0, $t0, 29

        li      $v0, PRINT_CHAR
        move    $a0, $t0
        syscall
```

Breakpoints (the red dot)!

- Set by clicking to the left of a line to pause execution there!
  - If column is missing, go to File > Preferences > Settings and turn on Glyph Margin
- Allow you to run the program using the button until a breakpoint is hit, and can then step through from there!
  - Prevents needing to manually step through large programs to get to a faulty section.

Translating MIPS: the Basics

Steps to Follow If Unsure How To Translate (1/3)

```
main:
        li      $v0, 0
        jr      $ra
```

1. Make sure to **set up the above** if it isn't in your file already!

- Without this, code can't run.
    - Need "*jr $ra*" to actually return from the main function
    - Need "*li $v0, 0*" to set return value to 0

Steps to Follow If Unsure How To Translate (2/3)

```c
#include <stdio.h>

int main(void) {
    int a, b, c;
    scanf("%d %d %d", &a, &b, &c);

    int num = (a + b) * (c - 2 / a);

    printf("The number generated from the inputs is %d. "
    "Was this expected?\n", num);

}
```

2. **Break down** more complex C statements into series of **single-operation C statements**

- Also **break down printf()** statements!
- Set up temp variables when necessary to mirror how you'll need to use temporary registers for calculations.

```c
#include <stdio.h>

int main(void) {
    int a;
    int b;
    int c;

    // Split up scanning as we can only scan in one int
    // per syscall!
    scanf("%d", &a);
    scanf("%d", &b);
    scanf("%d", &c);

    int num = a + b;
    int temp = 2 / a;
    temp = c - temp;        // temp = c - 2 / a;
    num = num * temp;       // num = (a + b) * (c - 2 / a);

    // Split up printf as printing strings + printing integers
    // require different syscalls!!
    printf("The number generated from the inputs is ");
    printf("%d", num);
    printf(". Was this expected?\n");

}
```

Steps to Follow If Unsure How To Translate (3/3)

3. **Use the <u>MIPS Documentation</u>** on the course website to look up syntax of the necessary operations!

   - **Instructions section** – details all available instructions, what they do exactly, and what the expected inputs are!
     - (Most important sections are Instruction + Description sections!)

   - **Syscall section** ("system services") – details the syscall numbers to be put in $v0, the arguments necessary and what registers to load them in, and where returns are placed.

	Instruction		Description
	CPU Arithmetic Instructions		
✓	ADD	R_d, R_s, R_t	$R_d = R_s + R_t$
✓	ADDI	R_t, R_s, Imm_{16}	$R_t = R_s + Imm_{16}$
✓	ADDU	R_d, R_s, R_t	$R_d = R_s + R_t$
✓	ADDIU	R_t, R_s, Imm_{16}	$R_t = R_s + Imm_{16}$
✓	SUB	R_d, R_s, R_t	$R_d = R_s - R_t$

System Services

back to top

The mipsy emulator provides a number of mechanisms for interacting with the host system, to provide input and output, file operations, and other miscellaneous services, which we refer to as "system calls" or "syscalls". These are invoked via the `syscall` instruction after storing the service code in the register `$v0`.

$v0 =	Arguments	Result	Description
	Printing Values		back to top
1	`$a0` : `int`		*print_int*: Print the integer in `$a0` to the console as a signed decimal.
2	`$f12` : `float`		*print_float*: Print the float in `$f12` to the console as a `%.8f`.
3	`$f12` / `$f13` : `double`		*print_double*: Print the double in `$f12` / `$f13` to the console as a `%.18g`
4	`$a0` : `char *`		*print_string*: Print the nul-terminated array of bytes referenced by `$a0` to the console as an ASCII string.
11	`$a0` : `char`		*print_character*: Print the character in `$a0`, analogous to *putchar*.

Translating Ifs and Loops

If Statements - Converting to Gotos

- Code with branching statements can be hard to think about converting to MIPS directly, especially complicated code
- Converting to **simple C using gotos** instead of if/else statements is a good way of figuring out what the structure of the labels in your C code will be!
- Need: **1 label per every if or else statement** (once broken down statements with || and && into several if statements), and **a label for all branches to rejoin at!**

```c
if (num % 2 == 0) {
    printf("The number is even.\n");
} else {
    printf("The number is odd.\n");
}

return 0;
```

```c
    if (num % 2 == 0) goto num_div_by_two_true;
    goto num_div_by_two_false;

num_div_by_two_true:
    printf("The number is even.\n");
    goto num_div_conditions_rejoin;

num_div_by_two_false:
    printf("The number is not divisible by 2 or 3.\n");
    goto num_div_conditions_rejoin;

num_div_conditions_rejoin:
    return 0;
```

If Statements - || and && (1/2)

- Remember – we can rewrite any combination of both as a series of if statements performing the same action inside (for logical OR) or nested if statements (logical AND)!! (covered in more depth in the lecture slides)

```
if (CONDITION_A || CONDITION_B) {
    // do action A
}
```

=

```
if (CONDITION_A) {
    // do action A
}
if (CONDITION_B) {
    // also do action A
}
```

```
if (CONDITION_A && CONDITION_B) {
    // do action B
}
```

=

```
if (CONDITION_A) {
    if (CONDITION_B) {
        // do action B
    }
}
```

If Statements - || and && (2/2)

- We can also use a similar trick to the one we use for logical OR statements to rewrite logical AND statements!
  - If there **isn't an "else"**, can instead **jump** straight to the **rejoining label**!
  - Can be more annoying to write than nested statements, but also results in code with fewer labels.
  - Which one works best depends on the situation!

```
if (CONDITION_A && CONDITION_B) {
    // do action 1
} else {
    // do action 2
}
```

=

```
if (!CONDITION_A) {
    // do action 2
} else if (!CONDITION_B) {
    // do action 2
} else {
    // do action 1
}
```

Loops in MIPS (1/2)

- Label schema (below) set out in the lectures is a good idea to follow for consistency!

```
main:
loop_name:              # label isn't necessary here, however having empty labels
                        # for organisation purposes can be a good idea!

loop_name_init:         # any loop-related initialisations, e.g setting i. Doesn't need sep label!
        li      $t0, 0  # int i = 0

loop_name_cond:                         # The condition to be checked - needs a separate label!!
        bge     $t0, 10, loop_name_end  # Need to reverse the while(condition A) - branch only when condition A fails!
                                        # e.g this is equivalent to while (i < 10)

loop_name_body:                         # Doesn't necessarily need separate label but good to have for consistency!
        # whatever happens in the body of the loop

loop_name_step:                         # Doesn't necessarily need separate label but good to have for consistency!
        addi    $t0, $t0, 1             # incrementing i
        j       loop_name_cond          # Don't forget to jump back to the condition!!

loop_name_end:                          # Outside of the loop - needs separate label so we can jump here!
        li      $v0, 0
        jr      $ra
```

Loops in MIPS (2/2)

THINGS TO NOTE

- MUST have separate labels above condition being tested and at the end of the loop – those labels are actually being jumped to!

- **Condition being tested** in the branch instruction **MUST be reversed EXACTLY** from what the condition in the while loop was!
  - E.g while (i < 100) must be translated to “bge $t0, 100, loop_end” (assuming i is in $t0), not “bgt $t0, 100, loop_end” as then the loop would run an extra time when i == 100!!